

PassiveSentinel CLI

Pipeline, runtime architecture and product design for the npm package @passivesentinel/cli (SIH 2026 PS-145)

Version 3, reviewed and corrected from v2 | 2 October 2026

Purpose. Define the command-line tool that wraps the trained PassiveSentinel model and turns passively observed traffic into standardized, explainable alerts. The CLI never retrains while detecting. It loads one versioned model bundle (feature schema, frozen preprocessing statistics, specialist models, fusion weights, thresholds and calibration) and runs the paper's L0 to L8 pipeline on it. The four paper constraints hold throughout: read-only ingest, no payload decryption, streaming with bounded latency, and a fixed alert schema.

Contents

0. Review of v2: what was wrong and what changed	2
1. Product overview	2
2. End-to-end architecture	3
3. Key design decisions	4
4. Package architecture	5
5. Runtime process model	7
6. CLI reference	8
7. What happens during analyze	9
8. Standard alert contract	10
9. Model bundle and manifest	11
10. Training-serving parity	11
11. Security and passivity	12
12. Benchmark mode	12
13. Implementation roadmap	13
14. Risks and open questions	13
15. Final product picture	13

0. Review of v2: what was wrong and what changed

The v2 structure (L0 to L9 layers, model bundle bound to the feature schema, the command list, the benchmark metrics) is sound and is kept. The review found presentation defects and several design gaps. Each is listed with its fix and where it is handled in this version.

#	Finding in v2	Impact	Fix in v3
1	Diagrams in Sections 2, 4 and 11 were drawn at a tiny scale: box text overlapped, labels ran into captions, and the package diagram printed some labels one letter per line.	High (readability)	All diagrams redrawn at full width with boxes sized to their text; checked page by page for overlap (Sections 2, 4, 6, 10).
2	The engine language was never stated. Training is Python; a Node CLI that reimplements L1 to L4 risks training-serving skew, which v2's own critical rule forbids.	High (correctness)	Decision D1 compares three engine options and recommends one (Section 3). Parity gate added (Section 10).
3	benchmark traffic.pcap cannot produce F1: a PCAP carries no labels.	High (validity)	Benchmark needs --labels, runs only on the held-out test split, and never tunes (Section 12).
4	listen was called receive-only, but nothing enforced it.	High (core constraint)	Passive-interface checks, a doctor command and localhost-only dashboard (Section 11).
5	No treatment of hostile input: the traffic is attacker-controlled, so the parsers are the attack surface of the monitoring host.	High (security)	Parser hardening, resource limits, bundle integrity (Section 11).
6	analyze-flow was described as 'start after parsing'. Flow records carry no DNS names, TLS fingerprints or packet sequences, so L5c and L5d cannot run on them.	Medium	Input capability matrix; disabled specialists are reported, not silent (Section 3).
7	Dashboard and storage were listed as FastAPI and DuckDB inside a Node package.	Medium	Node-native Fastify/ws (or served by the engine); bound to 127.0.0.1; storage optional (Section 4).
8	The alert contract was one unbroken line, evidence was empty, first_seen/last_seen from the paper were missing, and latency_ms was undefined.	Medium	Full schema, field table, severity bands and latency definition (Section 8).
9	No concurrency model, although the paper sets a throughput target and a latency budget.	Medium	Process model with sharding, bounded queues and drop counters (Section 5).
10	No config file, exit codes, bundle verification, platform notes or offline-install story.	Medium	Added (Sections 6 and 9).
11	The paper's 'sampled normal windows' for calibration monitoring did not appear.	Low	Added to Stage C (Section 2).
12	Package-name availability and GBM-to-ONNX conversion were not mentioned.	Low	Listed as open items (Section 14).

1. Product overview

PassiveSentinel is a developer-friendly security CLI. A user supplies a PCAP, an exported flow file or a receive-only stream; the runtime executes the L0 to L8 pipeline of the paper and writes machine-readable alerts, plus an optional terminal view, report and local dashboard.

```
passivesentinel analyze traffic.pcap
passivesentinel replay traffic.pcap --speed 10
passivesentinel analyze-flow flows.csv
passivesentinel listen --interface eth0
passivesentinel model info
passivesentinel benchmark flows_test.parquet --labels labels_test.csv
passivesentinel report alerts.jsonl
```

What the CLI is: a packaging and orchestration layer around a frozen, versioned model. What it is not: a trainer, an active scanner, or a blocker. Every recommended action is text for an analyst; the tool never executes mitigation.

2. End-to-end architecture

Figure 1 is the runtime pipeline. Stage A runs inexpensive logic on every flow or window. Stage B runs the DNS and TLS/QUIC specialists only where such records exist. Stage C runs fusion, calibration and alert generation for entities that Stage A or B flagged, plus a small sampled set of normal windows used to monitor calibration drift.

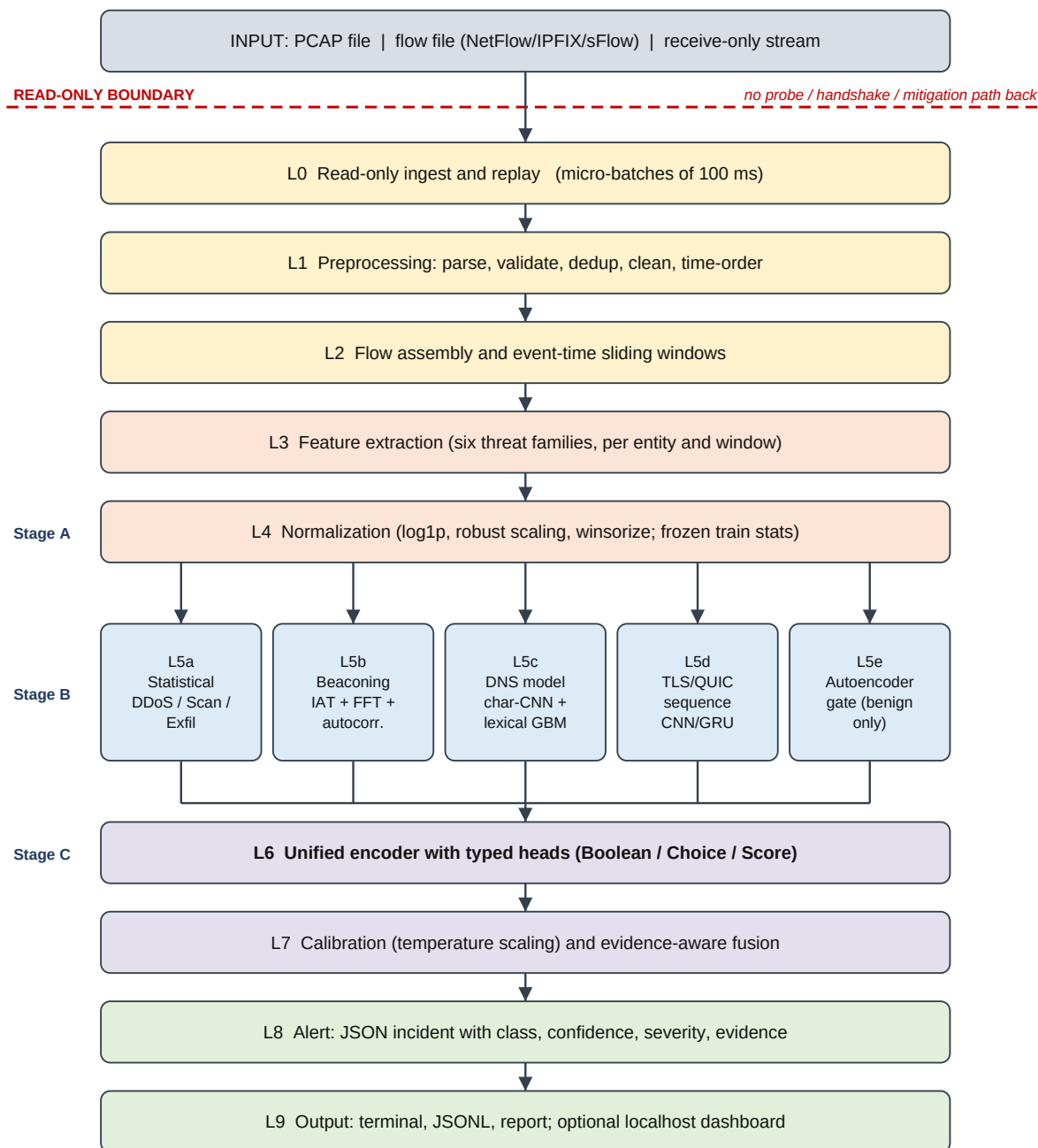


Figure 1. Runtime pipeline. The dashed red line is the passive boundary: nothing downstream of it can probe, handshake with, block or otherwise transmit toward the monitored network.

2.1 Layer responsibilities

Layer	Role	Runtime responsibility
L0	Read-only ingest / replay	PCAP, NetFlow/IPFIX/sFlow, receive-only stream; 100 ms micro-batches; bounded queue and visible drop counter.
L1	Preprocessing	Schema validation, UTC event time, de-duplication, direction canonicalization, numeric hygiene, late events, DNS name normalization, identifier firewall.
L2	Flows and windows	Canonical bidirectional flows; event-time sliding windows with watermark and bounded state.
L3	Features	Common plus DDoS, beaconing, DNS, TLS/QUIC, scan and exfiltration features.
L4	Normalization	Frozen training statistics: log1p, robust scaling, winsorization, encoders. Never refit at runtime.
L5a	Stat / GBM	DDoS, scan and exfiltration evidence.
L5b	Beaconing	Inter-arrival, FFT, autocorrelation, byte regularity.
L5c	DNS	Character CNN plus lexical/behavioural GBM for DGA and tunnelling.
L5d	TLS/QUIC	Metadata-only CNN/GRU plus handshake and fingerprint features.
L5e	Anomaly	Benign-only autoencoder gate for unknown behaviour.
L6	Fusion	Unified encoder with Boolean, Choice and Score heads.
L7	Calibration	Temperature scaling, evidence-aware confidence, incident coalescing.
L8	Alert	Fixed JSON incident: class, confidence, severity, evidence, latency.
L9	Output / dashboard	Terminal, JSONL, report; optional local dashboard on the same alert stream.

3. Key design decisions

3.1 D1: Where does the engine run?

The CLI wrapper is Node.js, but the model is trained in Python with its own L1 to L4 code. Whatever the engine is, the packaged runtime must reproduce the benchmarked model exactly. Three options:

Option	How	Pros	Cons
A. Node CLI + bundled Python engine	CLI spawns the same Python code used in training (packaged as a wheel or frozen binary) and talks to it over stdio JSON lines.	Zero training-serving skew; fastest to ship; scikit-learn / XGBoost / PyTorch used as trained.	Ships a Python runtime; two-language packaging; IPC overhead.
B. Pure TypeScript engine	Reimplement L1-L4 and L5 inference in TypeScript; models exported to ONNX and run with onnxruntime-node.	One npm install; no Python on the host.	Most work; every feature and scaler must be re-verified; GBM export adds risk; highest skew risk.
C. Native core (Rust/Go) via N-API	One native implementation used for both training export and runtime.	Fast; single implementation if training also uses it.	Largest effort; not realistic for first release.

Recommendation. Use Option A for 0.1.0, because the paper's critical rule (reuse the exact schema, statistics, thresholds and calibration) is satisfied by construction. Revisit Option B only if a single-binary, Python-free install becomes a requirement, and then only behind the parity gate of Section 10. This is a recommendation; confirm it before Phase 1.

3.2 D2: Model artifact formats

Neural models (L5c, L5d, L5e, L6) export to ONNX for portability. Tree models (XGBoost, LightGBM, scikit-learn) either stay in the Python engine (Option A) or convert with onnxmltools or skl2onnx (Option B), and the converted model must reproduce the original scores on the golden fixtures (Section 10). Scalers, vocabularies, thresholds and calibration are plain JSON with a sha256 recorded in the manifest.

3.3 D3: Input capability matrix

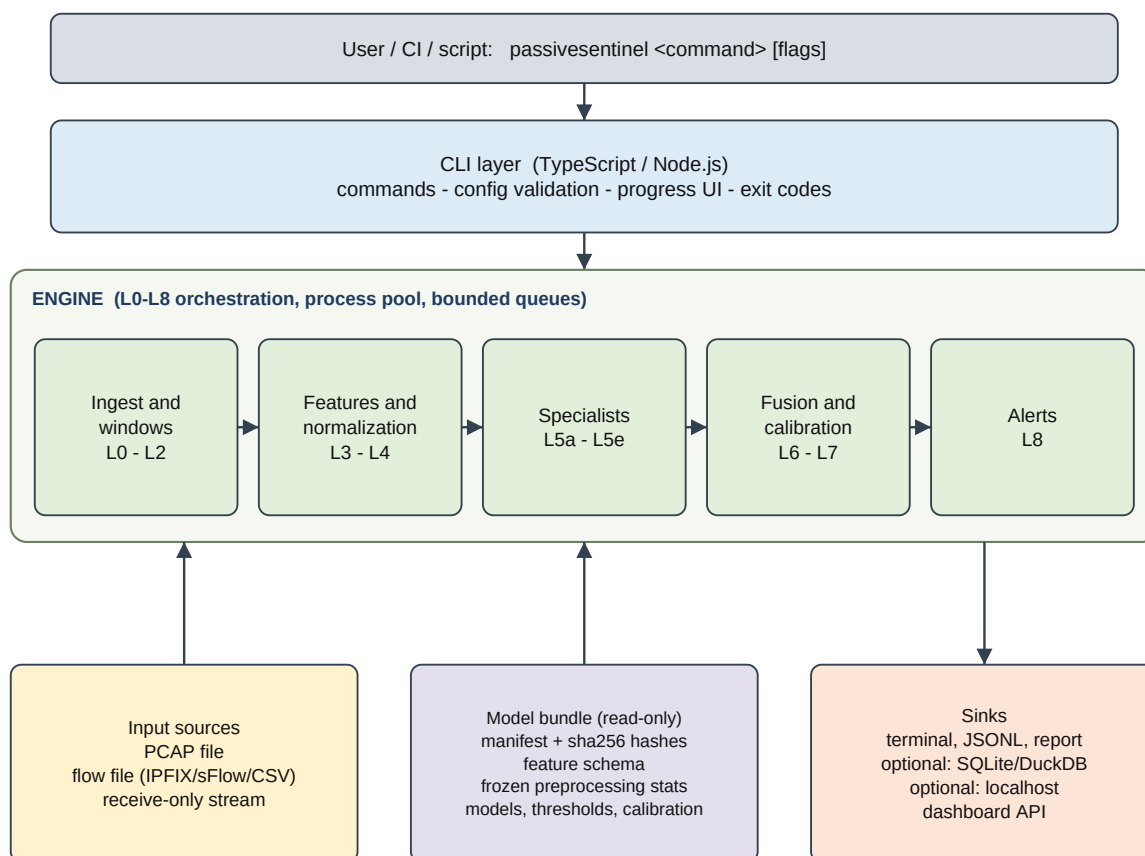
The specialists need different raw fields. A flow export does not contain DNS names, TLS handshakes or packet sequences, so some detectors cannot run on it. The CLI must say so rather than silently report 'no threats'.

Input	L5a stat	L5b beacon	L5c DNS	L5d TLS/QUIC	L5e anomaly
PCAP (full)	yes	yes	yes	yes (metadata only)	yes
Flow export (NetFlow/IPFIX/sFlow)	yes	yes	no (no names)	no (no handshake or sequence)	yes
Flow CSV (e.g. CIC-style)	yes (limited by columns)	yes	no	no	yes (feature set must match)
Flow file plus DNS and TLS side tables	yes	yes	yes	yes if sequences present	yes

A receive-only live stream follows the row of the source it carries (a packet capture behaves like a PCAP; a flow-export collector behaves like a flow export). The run summary lists every detector that was disabled for the input type, and each alert records the detectors that contributed to it.

4. Package architecture

The public package is a thin TypeScript layer over a reusable engine. Commands handle arguments, configuration, validation, progress display and output. The engine handles L0 to L8 orchestration.



Input and bundle are read-only. The engine opens no outbound connection.

Figure 2. Package components. Input and the model bundle are read-only; sinks are local.

4.1 Repository layout (corrected)

```
@passivesentinel/cli/
package.json  README.md  LICENSE
src/
  commands/    analyze, replay, listen, analyze-flow, model, benchmark,
               report, doctor
  engine/      client (spawns/talks to the engine), pipeline, scheduler
  config/      schema validation (JSON Schema), defaults
  alerts/      schema validation, severity bands, JSONL sink
  ui/          terminal renderer, progress, summary
  dashboard/   optional local API + web view (Fastify + ws), off by default
  engine/      Option A: Python engine (ingest, preprocess, windows, features,
               normalize, detectors, fusion, calibration) - same code as training
  models/
    v0.1/      manifest.json, preprocess/, specialists/, fusion/, thresholds.json
  test/        unit tests, golden fixtures, parity tests
```

4.2 Runtime and platform notes

- Target the current Node.js LTS (verify the exact version at release time). Live capture is supported on Linux only; offline commands may run on macOS and Windows.
- Native dependencies (ONNX runtime, packet-capture bindings, the Python engine under Option A) need a pinned, offline-installable distribution, because enclave hosts often have no internet access.
- The model bundle is versioned independently of the CLI. The CLI states which bundle versions it accepts (`min_cli_version` and schema hash in the manifest).
- The optional dashboard and any storage run on the same host, bind to 127.0.0.1 and are off by default.

5. Runtime process model

The paper sets a throughput target and a latency budget, so the runtime is multi-process rather than a single event loop. Work is sharded by entity key so that a window is never split across workers, and every hand-off is a bounded queue with a visible drop counter.

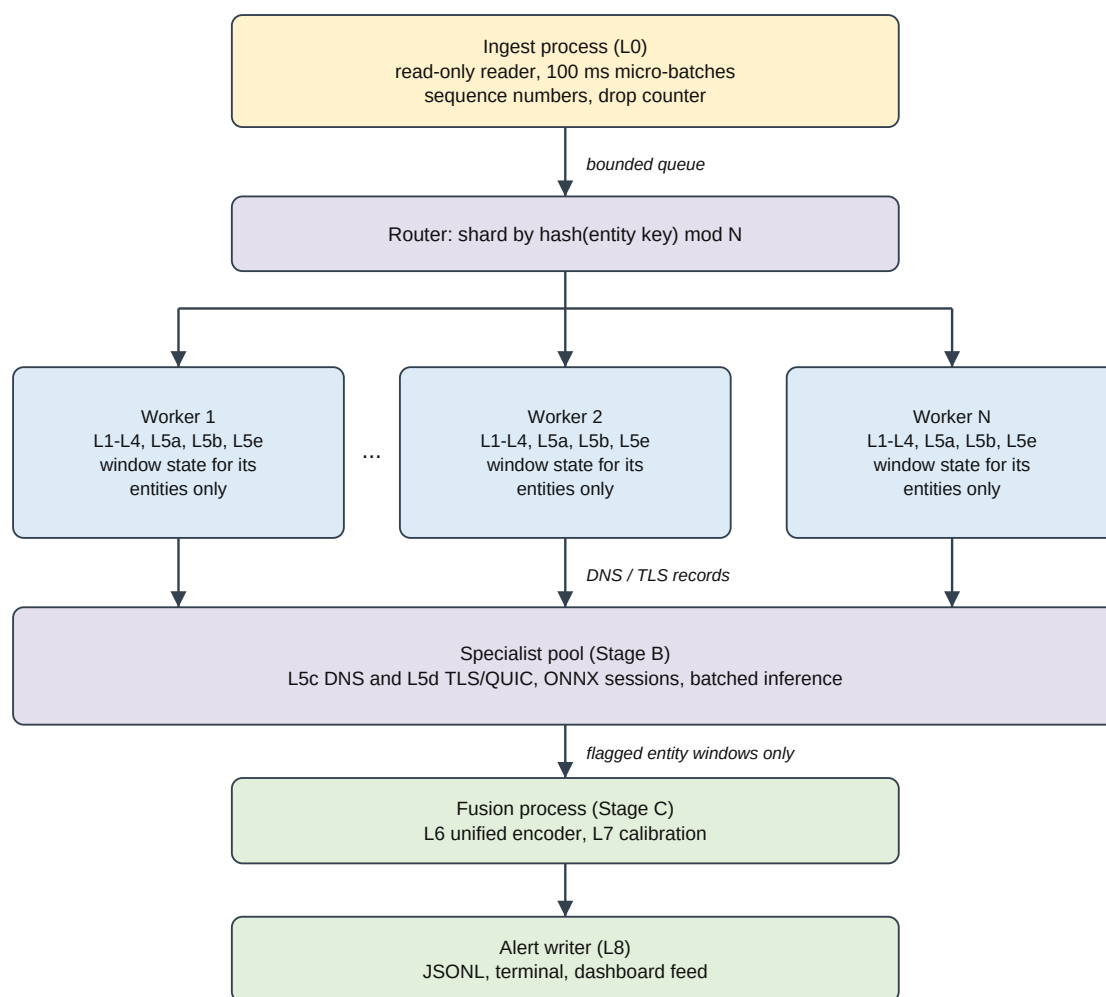


Figure 3. Process model. Each worker owns the window state of its entities, which keeps per-entity state local and memory bounded.

- **Sharding.** $\text{hash}(\text{entity key}) \bmod N$, where the key is destination (DDoS), source (scan, beaconing), internal host (exfiltration) or host plus apex domain (DNS). N defaults to CPU cores minus one (`--workers`).
- **Back-pressure.** A full queue increments a drop counter by reason and by shard. Records are never dropped silently, and the run summary reports drops. A run with drops is marked as exceeding its sustained rate.
- **Event time.** Windows close on a watermark (default 5 s lateness). Late records go to a side channel and are counted.
- **Bounded state.** Window state expires; unique-count and entropy structures use sketches (HyperLogLog, count-min), so memory is bounded under address-space-wide scans.
- **Determinism.** Fixed seeds per worker; replay of the same input with the same bundle gives the same alerts (alert ids and wall-clock fields excepted).
- **Shutdown.** SIGINT/SIGTERM flushes open windows, writes the run summary and exits with the code in Section 6.

6. CLI reference

6.1 Commands

Command	Input	Purpose	Output
analyze	PCAP	Offline detection through L0-L8	Terminal summary + JSONL
replay	PCAP or flow file	Same as analyze at original or scaled timing; used for demos and latency tests	Alerts + telemetry
analyze-flow	NetFlow/IPFIX/sFlow export or flow CSV	Enters at L1 and skips packet parsing; L5c/L5d run only if side tables are given	Alerts + disabled-detector list
listen	Receive-only interface or collector socket	Live passive monitoring; same engine	Live alerts + JSONL
model info	Bundle	Print version, schema hash, classes, thresholds, artifact hashes	Terminal / JSON
model verify (proposed)	Bundle	Recompute hashes and check schema compatibility	Exit code + report
benchmark	Labelled test data or capture + labels	Quality, calibration, latency, throughput, resources	benchmark.json + report
report	Alerts JSONL	Human-readable summary (counts by class and severity, timeline, top entities)	Markdown / HTML
doctor (proposed)	Host	Check capture permissions, interface passivity, bundle integrity, native deps	Terminal + exit code

6.2 Common flags

Flag	Meaning	Default
--model <path version>	Model bundle to load	latest installed
--output <file>	Write alerts as JSON lines	stdout summary only
--format pretty json jsonl	Terminal format	pretty on a TTY, jsonl otherwise
--min-severity low medium high critical	Hide alerts below this level	low
--min-confidence <0..1>	Hide alerts below this calibrated confidence	bundle default
--workers <N>	Worker processes	cores - 1
--disable <detector>	Turn off a detector (for ablation); recorded in output	none
--anonymize	Keyed hash of internal IPs in stored evidence	off
--speed <x max> (replay)	Timing scale	1
--labels <file> (benchmark)	Ground-truth labels for scoring	required
--strict-passive / --no-strict-passive (listen)	Enforce passive-interface checks	strict on
--config <file>	YAML or JSON config; flags override it	./passivesentinel.yaml if present
--quiet, --verbose, --no-color	Output control	-

6.3 Exit codes

Code	Meaning
0	Completed normally
1	Internal error
2	Usage or configuration error
3	Model bundle missing, corrupt, or incompatible with this CLI or schema
4	Input unreadable or unsupported
5	Passive-interface check failed (listen / doctor)
6	Dropped records exceeded the configured limit (--max-drop-rate)
10	Completed and alerts at or above the level set by --fail-on were produced (for CI use)

6.4 Example (illustrative output)

```
$ passivesentinel analyze traffic.pcap --output alerts.jsonl
PassiveSentinel 0.1.0 | bundle v0.1 (schema 9f3c1a) | workers 7
disabled for this input: none
[ingest] flows 1,204,311 | rejects 112 | late 41 | drops 0
[HIGH    ] recon_scan   conf 0.93  src-window 10s  840 distinct ports
[CRITICAL] ddos         conf 0.97  dst-window 1s   spoofed sources, 41k pps
Summary: 14 alerts (1 critical, 5 high, 8 medium) | e2e latency p95 412 ms
Alerts written to alerts.jsonl
```

7. What happens during analyze

#	Step	What happens
1	Load runtime	Read and verify the manifest, hashes, feature schema, frozen preprocessing statistics, thresholds and calibration. Refuse to run on any mismatch (exit 3).
2	Check the input	Detect the input type, apply the capability matrix (Section 3.3), and list disabled detectors.
3	Ingest	Read PCAP or flow data and assign sequence and receive metadata. No path toward monitored hosts is opened.
4	Clean	Validate, de-duplicate, normalize time to UTC, canonicalize direction, apply the identifier firewall, count rejects and late events.
5	Window	Assemble event-time flows and windows per entity with bounded state.
6	Features	Extract exactly the feature vector declared in the bundle's feature schema.
7	Normalize	Apply frozen training-time preprocessing. Never refit at runtime.
8	Stage A	Run L5a, L5b and L5e on the applicable stream.
9	Stage B	Run L5c on DNS records and L5d on TLS/QUIC records.
10	Stage C	Send flagged entity windows (and a small sample of normal windows for calibration monitoring) to L6 to L8.
11	Fuse and calibrate	L6 produces unified scores; L7 applies temperature scaling and evidence-aware confidence.
12	Alert	L8 creates or coalesces the incident and writes JSONL and terminal output, with latency_ms.
13	Summarize	Print counts, rejects, drops, disabled detectors and latency percentiles. Optional L9 consumes the same stream.

8. Standard alert contract

Every alert is one JSON object per line (UTF-8), validated against the bundle's JSON Schema before it is written.

```
{
  "schema_version": "1.0",
  "alert_id": "a-000123",
  "timestamp": "2026-10-02T10:15:30.480Z",
  "first_seen": "2026-10-02T10:15:20.000Z",
  "last_seen": "2026-10-02T10:15:30.000Z",
  "flow_id": "src:10.2.3.4|win:10s|t:1790935720",
  "threat_class": "recon_scan",
  "confidence": 0.93,
  "severity": {"level": "high", "score": 7.8},
  "evidence": {
    "distinct_dst_ports": {"value": 840, "baseline": 6, "robust_z": 41.2},
    "failed_conn_ratio": {"value": 0.91, "baseline": 0.04},
    "detector_scores": {"stat": 0.97, "autoencoder": 0.81, "unified": 0.93}
  },
  "detectors_used": ["L5a", "L5e", "L6"],
  "recommended_action": "Analyst review only",
  "latency_ms": 412
}
```

Field	Meaning
schema_version	Version of this contract; the bundle pins it
alert_id	Unique within a run
timestamp	Alert emission time (UTC). first_seen and last_seen are event time of the incident
flow_id	Triggering flow id, or the entity-window key for window-level alerts
threat_class	ddos, beaconing, dga_tunnel, encrypted_malware, recon_scan, exfiltration, or anomaly
confidence	Calibrated probability of the class, 0 to 1
severity	Numeric score 0 to 10 and level. Proposed bands: low < 4, medium 4 to < 7, high 7 to < 9, critical 9 and above (configurable in the bundle)
evidence	Readable supporting signals: deviations from the benign baseline plus detector-specific facts (period, ports, domain, fingerprint). Never raw payload
detectors_used	Which detectors contributed; shows when a specialist was disabled by input type
recommended_action	Analyst text only; never executed
latency_ms	Alert emission time minus the closing time of the triggering window, on the replay clock in replay mode

9. Model bundle and manifest

The manifest binds a model version to the exact artifacts it was evaluated with. The CLI verifies it at load time and refuses to run on any mismatch, which prevents silently running incompatible preprocessing or weights.

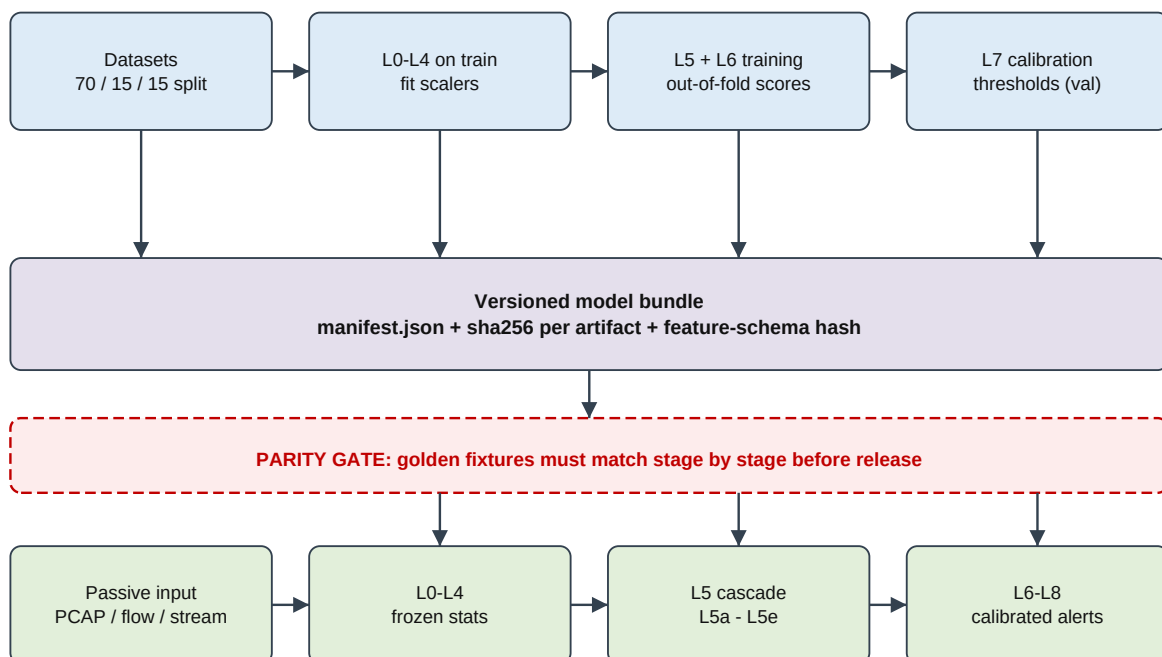
```
{
  "bundle_version": "0.1.0",
  "created_utc": "2026-10-02T00:00:00Z",
  "min_cli_version": "0.1.0",
  "alert_schema_version": "1.0",
  "feature_schema": {"file": "features.yaml", "sha256": "..."},
  "preprocess": [{"file": "preprocess/scaler.json", "sha256": "..."}],
  "models": [
    {"id": "L5c_dns_cnn", "format": "onnx",
     "file": "specialists/dns_cnn.onnx", "sha256": "..."},
    {"id": "L6_fusion", "format": "onnx",
     "file": "fusion/unified.onnx", "sha256": "..."}
  ],
  "thresholds": {"file": "thresholds.json", "sha256": "..."},
  "calibration": {"temperature": 1.37, "file": "fusion/calibration.json"},
  "label_map": ["benign", "ddos", "beaconing", "dga_tunnel",
                "encrypted_malware", "recon_scan", "exfiltration"],
  "training": {"split_manifest_sha256": "...", "seeds": [1,2,3,4,5]},
  "window_config": {"ddos_s": 1, "scan_s": 10, "exfil_s": 60, "watermark_s": 5}
}
```

The values above are placeholders. The real numbers (temperature, thresholds, hashes) come from the training run and are never edited by hand.

10. Training-serving parity

The paper's critical rule is that the CLI must reuse the exact schema, statistics, thresholds and calibration of the evaluated model. Figure 4 shows where that is enforced.

TRAINING (offline)



RUNTIME (CLI): same schema, frozen statistics, never refits

Figure 4. Training produces the bundle; the parity gate proves the runtime reproduces it.

Golden fixtures. Pick a small, representative set of captures and flow files covering all six threat families and benign traffic. Run the training pipeline offline and store, per fixture, the L3 features, L4 normalized values, L5 scores, L6 outputs and final alerts. The CLI test suite replays the same fixtures and compares stage by stage.

Acceptance criteria (proposed). Feature and normalized values agree within a small numeric tolerance (for example 1e-5 relative for float32); predicted class agrees on every fixture window; calibrated confidence differs by less than 0.001; alert counts match exactly. The release pipeline fails if any stage disagrees. Under Option A the same code runs, so the gate is a regression test; under Option B it is the primary safeguard.

11. Security and passivity

The monitored traffic is attacker-controlled, and the monitoring host is itself a target. The following measures are defence in depth. The strongest guarantee of one-way flow is physical (a TAP or data diode with no transmit path); software checks supplement it and do not replace it.

Area	Measure
Passive interface (listen)	Capture without any injection call; the interface should have no IP address and no route; drop elevated privileges after opening the capture where the platform allows; doctor and --strict-passive check these and exit 5 on failure. Running with the checks disabled is recorded in the run metadata.
No outbound traffic	The engine opens no outbound connection; no telemetry, no update check, no model download at run time. Recommended deployment: run in a network namespace or container with no route, and verify with doctor.
Hostile input	Parse in worker processes with memory and time limits; cap packet, name and record sizes; reject malformed records and count them; fuzz the PCAP, DNS and TLS parsers; never pass traffic-derived strings to a shell, path or eval.
Bundle integrity	sha256 per artifact in the manifest, verified at load; optional signature verification for distributed bundles; refuse on mismatch (exit 3).
Dashboard / API	Off by default; binds to 127.0.0.1; optional access token; read-only view of the alert stream; no endpoint can change the model or send traffic.
Output hygiene	Escape or strip control characters in domain names and other attacker-controlled text before printing; alert files created with owner-only permissions; --anonymize hashes internal IPs in stored evidence.
Supply chain	Pinned dependencies, lockfile, SBOM, and an offline-installable release.

12. Benchmark mode

The benchmark measures detection quality and runtime behaviour. Quality needs labels, so the command takes a labelled test set (flows or a capture plus a labels file). It evaluates only the held-out test split, with the thresholds already stored in the bundle; it never tunes anything. Throughput is measured by replaying the same data at stepped offered rates.

Area	Metrics
Detection	Macro-F1, per-class precision/recall/F1, PR-AUC, MCC
Calibration	ECE and Brier score before and after calibration
Operations	False-positive rate, alerts per hour on benign replay, episode recall
Latency	Time-to-detect p50/p95; end-to-end alert latency p50/p95/p99
Throughput	Offered flows/s, achieved flows/s, dropped flows (replay at 1k, 5k, 10k, 20k, 30k, 50k)
Resources	CPU utilization and peak RSS memory
Generalization	Cross-dataset macro-F1 and a site-specific benign replay

- Report the hardware, software versions, bundle version, dataset hashes and seeds with every result file.
- When the input is a PCAP, report the packet exporter and the pipeline separately, so a slow exporter is not mistaken for a slow model.
- Any run with dropped records is marked as exceeding the sustained rate.

- **Paper design allocation (targets, not claims):** L0 ingest 100 ms p95; L1 to L4 200 ms; L5 detectors 300 ms; L6 to L8 400 ms; end-to-end target of at most 1 second after window close. The benchmark exists to measure whether the implementation meets them.

13. Implementation roadmap

Phase	Deliverable	Acceptance test
0	Decisions D1 to D3 confirmed; manifest and alert schemas frozen	Written decision record; schemas validate fixtures
1	Bundle loader; <code>model info</code> and <code>model verify</code>	Corrupt or incompatible bundle exits 3
2	L0 to L2 ingest and window engine with PCAP and flow fixtures	Window and watermark unit tests pass
3	L3 to L4 features and normalization match training	Parity gate: features and normalized values match goldens
4	L5 specialist inference	Scores match goldens; disabled-detector reporting works
5	L6 to L8 fusion, calibration, JSONL alerts	Alerts match goldens; schema validation 100 percent
6	Stable analyze, replay, analyze-flow, model, report commands; exit codes	CLI end-to-end tests; exit-code tests
7	Benchmark command	Reproduces offline test metrics within tolerance; throughput table filled
8	Receive-only listen and doctor	Passive checks fail on an interface with an IP; run in a no-route namespace
9	Optional dashboard / API	Localhost-only; renders the live alert stream

First release: `@passivesentinel/cli 0.1.0` with offline analysis, replay, exported-flow analysis, model inspection and benchmark mode. Add live receive-only monitoring after the offline path is stable and the parity gate is green.

14. Risks and open questions

Item	Risk or question	Mitigation / next step
Engine option	Option choice changes packaging and effort	Confirm D1 before Phase 1
Package name	<code>@passivesentinel</code> scope and package name may be taken	Check npm availability before publishing
GBM export	Tree-model conversion to ONNX can change scores	Keep trees in the engine (Option A) or gate on goldens
Flow-only inputs	DNS/TLS detectors unavailable; coverage lower	Capability matrix and disabled-detector reporting; prefer PCAP for full coverage
QUIC and TLS 1.3	Less handshake metadata visible over time	Sequence-based features; report coverage honestly
Performance in Node	Single process cannot meet the throughput target	Process pool and sharding; measure, then size N
Dataset realism	Lab data overstates field accuracy	Cross-dataset and site-specific benign replay in benchmark
Severity labels	Severity is rule-derived, not analyst-labelled	Present as prioritization aid; allow band tuning

15. Final product picture

The final product is a passive streaming detection engine distributed through npm. One runtime serves offline PCAP analysis, replay, exported flows and receive-only live monitoring.

Training side: datasets (70/15/15 time-ordered split), L0 to L4 preparation with scalers fitted on train only, specialist models, out-of-fold scores, L6 fusion, L7 calibration, then a versioned bundle.

Runtime side: passive input, L0 to L4 with frozen statistics, the L5 cascade, L6 to L8 for suspicious entities, JSON alerts, then terminal, report or dashboard.

Critical rule: the CLI must use the exact feature schema, preprocessing statistics, thresholds and calibration artifacts of the evaluated model, and the parity gate (Section 10) is what proves it. Otherwise the packaged runtime is not reproducing the benchmarked model.